About Cassandra

What will be covered

- About Cassandra
- Data Types in Cassandra
- Limitations of Cassandra

About Cassandra

OBJECT™ TECHNOLOGIES

- Cassandra is a distributed database management system designed for handling a high volume of structured data .
- Cassandra handles the huge amount of data with its distributed architecture.
- Data is placed on different machines with more than one replication factor that provides high availability and no single point of failure.

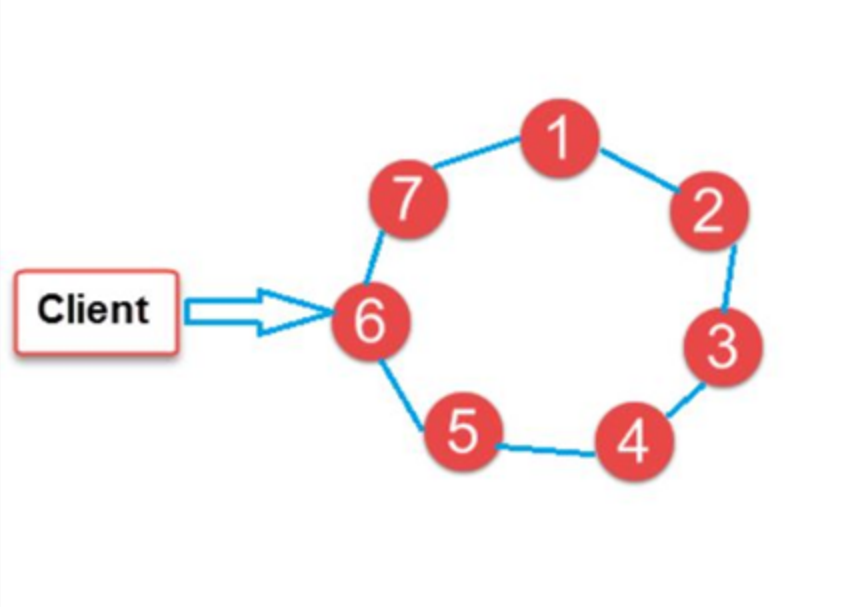
Limitations of Cassandra

OBJECT™ TECHNOLOGIES

- Group functions are not available in Cassandra.
- Cassandra dose not support ‘Joins’. Hence data normalization fails and we need to denormalise the data.
- search only on key value
- Data consistancy is not taken care of.
- Uses AID properties.Where consistancy is not taken by Cassandra database.
- Deletion requires whwer clause.

Cassandra Architecture

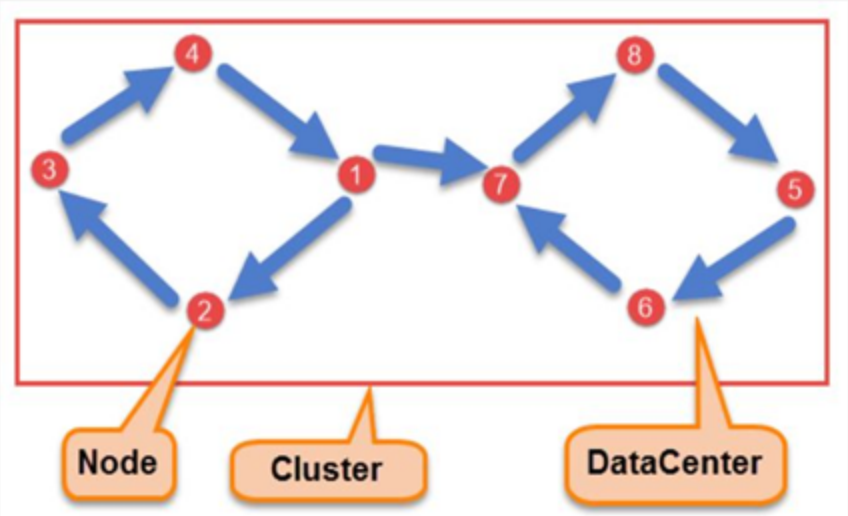
In the image below, circles are Cassandra nodes and lines between the circles shows distributed architecture, while the client is sending data to the node.



Cassandra is designed to handle Big Data. Cassandra's main feature is to store data on multiple nodes with no single point of failure.

The reason for this kind of Cassandra's architecture was that the hardware failure can occur at any time. Any node can be down. In case of failure data stored in another node can be used. Hence, Cassandra is designed with its distributed architecture.

Components of Cassandra Architecture



Cluster

The cluster is the collection of many data centers.

Commit Log

Every write operation is written to Commit Log. Commit log is used for crash recovery.

Node

Node is the place where data is stored. It is the basic component of Cassandra.

Data Center

A collection of nodes are called data center. Many nodes are categorized as a data center.

Mem-table

After data is written in Commit log, data is written in Mem-table. Data is written in Mem-table temporarily.

SSTable

When Mem-table reaches a certain threshold, data is flushed to an SSTable disk file.

Keyspace in Cassandra

OBJECT™ TECHNOLOGIES

Syntax ::

create keyspace hr with replication=

{‘class’:’SimpleStrategyable’, ‘replication_factor’:1};

Explaination ::

- keyspace is equivalent to database.
- Above command creates a keyspace.
- Once a keyspace is created we can create cloumn families.
- Replication Factor determines total number replicas placed on different nodes.
- For ensuring there is no single point of failure, replication factor must be three.
- As hardware problem can occur or link can be down at any time during data process, a solution is required to provide a backup when the problem has occurred. So data is replicated for assuring no single point of failure.

SimpleStrategy in Cassandra

- SimpleStrategy is used when you have just one data center.
- SimpleStrategy places the first replica on the node selected by the partitioner.
- After that, remaining replicas are placed in clockwise direction in the Node ring.

What will be covered

- Collections in Cassandra
- Creating Table
- Inserting Rows
- Updating Rows
- Add/Delete Columns
- Creating Index

Collections in Cassandra

OBJECT™ TECHNOLOGIES

Cassandra supports multiple values to be stored in the column.

Following are the collections available

- set
  - group of unique values of same type
- list
  - group of collections ,ordered and duplicates are allwed.
- tuple
  - Fixed length, list of diff datatypes.
  - Duplicates allowed
- map
  - set of key-value pair where key is unique.

Creating Database

Syntax ::

create keyspace hr with replication={‘class’:’SimpleStrategy’, ‘replication_factor’:1};

Explaination ::

- Creates database hr.
- Single server is ‘simplestrategy’
- Multiple server is ‘complexstrategy’

Syntax ::

use hr;

Creating Table

Syntax ::

```
create table book
(
  bkid int ,
  bkname varchar(50),
  bkauthor varchar(15)
);
```

Inserting values

```
insert into book
values(1,'Shiva Triology','Amish Tripathi');
```

```
create table student
(
  rollno int,
  name text,
  course text,
  hobbies set<text>,
  dept tuple<int,text,int>
  marks map <text,int>,
  primary key(roll)
);
```

here tuple <int,text,int> means 3 values

1 st int

2 nd text

3rd int

map<text,int>
text is subject
int is marks

www.object.co.in

Creating tables using collections

OBJECT™ TECHNOLOGIES

```
insert into student(roll,name,hobbies,marks,dept)
values(12,'Rajan',{'reading','biking'},{'Java':99,'Python':98,'C++':99},
(12,'HR','Mumbai'));

select * from student;
```

Note: Column list is compulsory.

Updating the date

Query ::

Update hobbies of Rajan and add a new hobby as swimming.

Solution::

update student

set hobbies =hobbies +{‘swimming’}

where roll=123;

or

update student

set hobbies =hobbies +{‘swimming’,’gaming’};

where roll =123;

+ : addition

- : subtraction

Not in tuple , since it is a fixed size.

www.object.co.in

User defined data type

```
create type friend (num int,name varchar , mobiletext);

cerate table mytable
( num int primary key,
  finfo friend);

insert into mytab(num,finfo)
values(123,{num:111,name:'Rajesh',mobile:'111111111'});
```

Alter table

Add a column::

alter table mytab

add column varchar;

Droping a column::

alter table mytab

drop column ;

Renaming column::

-Can rename only primary key column

alter table mytab

rename num to fid;

Truncate table::

truncate tablename

www.object.co.in

Index in Cassendra

Creating index ::

```
create index idx_name
on student(name);
```

Deleting index ::

```
drop index idx_name;
```

Batch statements

```
begin batch
  <insert>
  <delete>
  <update>
apply batch
```

Batch Statements

```
begin batch
  insert into student(rollno,name,hobbies)
  values(10,'Yash',{'Reading','Singing'});

  update student
  set course ='BSC'
  where rollno = 100;

  delete from student
  where rollno=120;
apply batch;
```